Mazda RAG Chatbot: A Retrieval-Augmented Generation System

Author: Rahji Reed
Course: AGAI-03

Date: May 2026

Abstract

This project presents a Retrieval-Augmented Generation (RAG) system designed to answer questions about Mazda vehicles, technologies, and historical automotive information. The system integrates web scraping, text preprocessing, embedding generation, vector database storage, semantic retrieval, and LLM-based answer generation. Additionally, a synthetic Q/A dataset of 100–200 high-quality question-answer pairs was generated to support evaluation and dataset enrichment. A Streamlit interface provides an interactive chat experience. This report documents the full pipeline, implementation details, challenges, and results.

Table of Contents

1. Introduction

Large Language Models (LLMs) are powerful but limited by their training data and inability to access up-to-date or domain-specific information. Retrieval-Augmented Generation (RAG) solves this by combining:

- A **retriever** that finds relevant documents
- A **generator** (LLM) that produces grounded answers

This project builds a complete RAG pipeline focused on Mazda automotive knowledge, using scraped Wikipedia pages as the knowledge base.

2. System Overview

The system consists of six major components:

- **Scraper:** Downloads Mazda-related Wikipedia pages
- **Chunker:** Splits text into overlapping segments
- **Embedder:** Converts chunks into vector embeddings
- **Vector Store:** Stores embeddings in ChromaDB
- **Retriever:** Finds relevant chunks for a user query
- **RAG Generator:** Produces grounded answers using Llama 3
- **Synthetic QA Generator:** Creates 100–200 Q/A pairs
- **Streamlit UI:** Provides an interactive chat interface

3. Phase 1 — Data Acquisition (Web Scraping)

Wikipedia pages were selected for:

- Mazda CX-5
- Mazda CX-50
- Mazda CX-90
- Mazda3
- Mazda6
- Mazda MX-5
- Mazda RX-7
- Mazda rotary engine
- Skyactiv technology

Scraping was performed using:

- `requests`
- `BeautifulSoup4`

Each page was cleaned and saved as a `.txt` file in `data/txt/`.

4. Phase 1 — Text Cleaning & Chunking

Raw scraped text is too long for embedding models, so it was split into overlapping chunks:

- **Chunk size:** 500–1000 characters
- **Overlap:** 100–200 characters

This improves retrieval accuracy by preserving context across boundaries.

Chunks were saved in `data/chunks/`.

5. Phase 2 — Embedding Generation

Embeddings were generated using:

Model: `sentence-transformers/all-MiniLM-L6-v2`

Reasons for choosing this model:

- Fast
- Lightweight
- High semantic accuracy
- Works well with ChromaDB

Each chunk was converted into a 384-dimensional vector.

6. Phase 3 — Vector Database (ChromaDB)

ChromaDB was used as the persistent vector store.

Advantages:

- Fast similarity search
- Persistent storage
- Easy integration with Python
- Supports metadata (e.g., source page)

Each chunk was stored with:

- `id`
- `text`
- `embedding`
- `source_page`

7. Phase 4 — Semantic Retrieval

When a user asks a question:

1. The query is embedded
2. ChromaDB performs a similarity search
3. Top-k chunks (usually 3–5) are returned
4. These chunks become the context for the LLM

This ensures answers are grounded in Mazda-specific knowledge.

8. Phase 5 — RAG Answer Generation

The RAG pipeline uses:

- **Retriever:** ChromaDB
- **Generator:** Llama 3 (via Ollama)

Prompt structure:

Code
Use ONLY the context below to answer the question.
If the answer is not in the context, say "I don't know."

Context:
[retrieved chunks]

Question:
[user question]

This prevents hallucinations and ensures factual accuracy.

9. Phase 6 — Synthetic Q/A Dataset Generation

A synthetic dataset of **100–200 question-answer pairs** was generated using:

- Llama 3
- Cleaned Mazda text chunks
- Strict JSON formatting
- Retry logic for malformed JSON

Each Q/A pair includes:

- `question`
- `answer`
- `source_page`

The dataset is saved as:

- `qa_dataset.csv`
- `qa_dataset.json`

This dataset can be used for:

- Evaluation
- Fine-tuning
- Reranker training
- Demonstrating dataset creation

10. Streamlit User Interface

A simple Streamlit app (`streamlit_app.py`) provides:

- A chat interface
- Real-time RAG responses
- Display of retrieved context
- Clean, user-friendly layout

Run locally with:

Code
streamlit run streamlit_app.py

11. Results & Evaluation

The system successfully answers questions such as:

- “What is Skyactiv technology”
- “Explain the Wankel rotary engine”
- “Differences between RX-7 and RX-8”
- “Tell me about the Mazda CX-50”

Evaluation metrics:

- **Accuracy:** High for factual Mazda questions
- **Hallucination rate:** Low due to strict grounding

- **Retrieval quality:** Strong due to chunking + MiniLM embeddings

12. Challenges & Limitations

- Llama 3 sometimes outputs invalid JSON
- Local LLM inference is slower than cloud models
- Wikipedia scraping may include noise
- Some Mazda topics have limited publicly available text

13. Future Improvements

- Switch to **Groq** for faster Q/A generation
- Add a **reranker** (cross-encoder)
- Add **hybrid retrieval** (BM25 + embeddings)
- Add **confidence scores** in the UI
- Deploy the Streamlit app online

14. Conclusion

This project demonstrates a complete RAG pipeline built from scratch, including scraping, chunking, embedding, retrieval, LLM generation, and synthetic dataset creation. The system performs well on Mazda-related queries and provides a strong foundation for future improvements such as reranking, fine-tuning, and deployment.

15. References

- Wikipedia (Mazda pages)
- SentenceTransformers documentation
- ChromaDB documentation
- Streamlit documentation
- Ollama documentation